

Dirty Checking and Flush

JPA / Hibernate Internals for Beginners

Sub-Chapter 04.08 | Chapter 04: JPA and Hibernate

Dirty Checking	Auto-detect field changes without explicit save()
How It Works	Snapshot at load, compare at flush
FlushMode	AUTO, COMMIT, MANUAL
Flush Triggers	Before query execution, before commit
@DynamicUpdate	Only update changed columns
Explicit flush()	flush() vs relying on commit
Detached and merge()	Re-attach detached entities

1 Dirty Checking

Auto-detect changes without explicit save()

Dirty Checking is one of Hibernate's most powerful features. When an entity is loaded inside an active persistence context (EntityManager / Session), Hibernate keeps an internal copy of its original state called a **snapshot**. At flush time, Hibernate compares the current state with the snapshot. If any field has changed, it automatically generates and executes an UPDATE SQL statement -- no explicit em.persist() or save() call is needed. This makes entity mutation feel natural and transparent.

```
// No explicit save/update call needed
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

User user = em.find(User.class, 1L); // snapshot stored
user.setEmail("new@example.com");   // mark as dirty

em.getTransaction().commit();        // flush -> UPDATE fired
em.close();
```

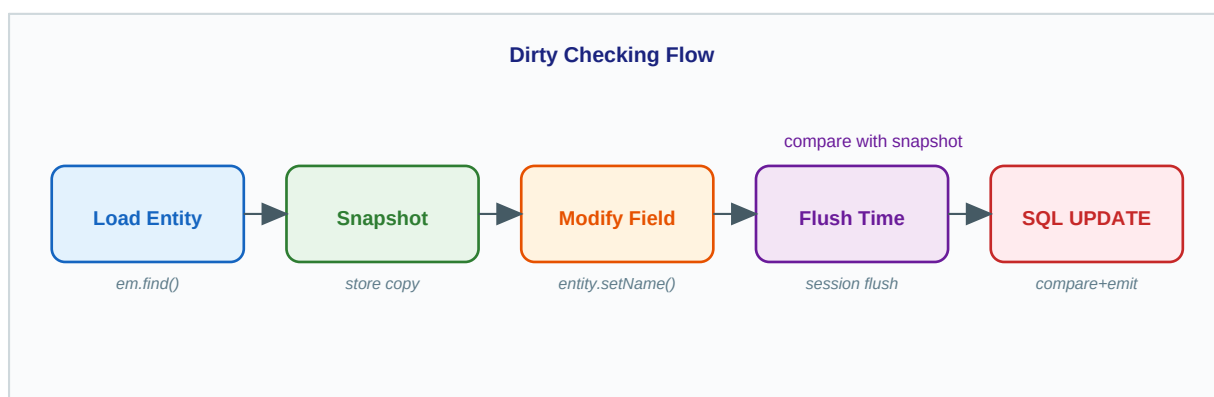


Figure 1.1 -- Dirty Checking Flow: entity loaded, snapshot stored, field modified, flush compares, UPDATE generated.

Key Rules

- Only managed entities (inside the persistence context) are dirty-checked.
- Detached or transient entities are NOT dirty-checked.
- Dirty checking fires at flush time, not immediately after the field is set.
- Even if you set the same value, Hibernate still fires the UPDATE (unless @DynamicUpdate is used).

Q1. What is dirty checking in Hibernate?

It is the automatic detection of field changes on managed entities. Hibernate compares the current state to the snapshot taken at load time and issues an UPDATE if any field differs.

Q2. Do you need to call save() after changing an entity field?

No. If the entity is managed (inside an active EntityManager), the change is picked up automatically at flush time.

Q3. When does the snapshot get stored?

The snapshot is stored when the entity is first loaded via em.find(), a JPQL query, or em.persist() for a new entity.

Q4. Does dirty checking work outside a transaction?

No. Dirty checking and flushing require an active transaction to send changes to the database.

Q5. What happens if you set the same value on an entity field?

By default Hibernate still fires an UPDATE because it does a shallow comparison. Use `@DynamicUpdate` to avoid updating unchanged columns.

2 How It Works

Snapshot at load time, compare at flush time

When `em.find()` (or a JPQL query) returns an entity, Hibernate stores an array of the entity's property values as a snapshot inside the first-level cache (the Session cache). This snapshot is a shallow copy of the original values. At flush time, Hibernate iterates over all managed entities, reads their current field values, and compares them element-by-element with the snapshot. Any entity where at least one field differs is considered **dirty**, and an UPDATE statement is scheduled for it.

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

Product p = em.find(Product.class, 42L);
// Hibernate now holds: snapshot = { name="Widget", price=9.99 }

p.setPrice(12.50);
// Current state:          { name="Widget", price=12.50 }

// At commit -> flush: price differs
// UPDATE product SET price=12.50 WHERE id=42
em.getTransaction().commit();
em.close();
```

Key Rules

- The snapshot is stored per-entity inside the first-level (L1) cache.
- Comparison is field-by-field using `equals()` for objects and `==` for primitives.
- Collections are tracked separately via Hibernate's `PersistentCollection` wrappers.
- Calling `em.clear()` discards all snapshots -- entities become detached.
- The L1 cache is always enabled and scoped to the `EntityManager` / `Session`.

Q1. Where does Hibernate store the snapshot?

Inside the first-level cache (L1 cache / persistence context), as an array of property values.

Q2. How does Hibernate compare old and new values?

Field-by-field: `equals()` for object types, `==` for primitives. For collections, it uses `PersistentCollection` wrappers that track additions and removals.

Q3. What happens to the snapshot after a flush?

The snapshot is updated to reflect the newly flushed state so subsequent changes within the same transaction are still tracked correctly.

Q4. What does `em.clear()` do to snapshots?

It evicts all entities from the L1 cache, discarding their snapshots. Evicted entities become detached and are no longer dirty-checked.

Q5. Can Hibernate detect changes inside an embedded object?

Yes. Hibernate tracks embedded (`@Embeddable`) objects as part of the owning entity's snapshot and includes them in the field-by-field comparison.

3 FlushMode

AUTO, COMMIT, MANUAL

FlushMode controls **when** Hibernate synchronises the persistence context with the database. Choosing the right mode can significantly affect performance and correctness. The default mode is AUTO, which balances safety and efficiency for most applications.

```
EntityManager em = emf.createEntityManager();
Session session = em.unwrap(Session.class);

session.setHibernateFlushMode(FlushMode.COMMIT); // only flush at commit
session.beginTransaction();

Order o = session.get(Order.class, 7L);
o.setStatus("SHIPPED");

// No auto-flush before JPQL queries now
session.getTransaction().commit(); // flush happens here
em.close();
```

FlushMode Comparison		
FlushMode	When Flush Happens	Typical Use Case
AUTO	Before query exec + before commit	Default; safe for most apps
COMMIT	Only before transaction commit	Read-heavy; skip mid-tx flush
MANUAL	Only on explicit em.flush()	Batch jobs; full manual control

Figure 3.1 -- FlushMode comparison: when each mode triggers a flush and its typical use case.

Key Rules

- AUTO is the safe default -- flushes before queries so you see your own changes.
- COMMIT skips pre-query flushes -- risk of reading stale data within the same transaction.
- MANUAL requires em.flush() before every query that must see pending changes.
- Always flush before native SQL queries; Hibernate cannot analyse them for staleness.

Q1. What is the default FlushMode in Hibernate?

FlushMode.AUTO. It flushes pending changes before any JPQL or Criteria query that might be affected by those changes, and always before a commit.

Q2. When would you use FlushMode.COMMIT?

In read-heavy operations where you want to avoid unnecessary flushes mid-transaction. You accept the risk that queries inside the transaction may not see un-flushed changes.

Q3. What is FlushMode.MANUAL used for?

Batch processing or reporting sessions where you want complete control over when SQL is sent to the database. You must call session.flush() manually.

Q4. Can FlushMode be changed at runtime?

Yes. You can call session.setHibernateFlushMode() at any point during a session.

Q5. Does FlushMode.AUTO always flush before every query?

No. It only flushes before queries that Hibernate determines could be affected by the pending dirty entities (based on the query's target tables).

4 Flush Triggers

Before query execution and before commit

Even with the default `FlushMode.AUTO`, flush is not triggered after every change. Hibernate batches changes and flushes them at specific well-defined moments: (1) just before executing a JPQL/Criteria query that targets the same table as a dirty entity, and (2) just before the transaction commits. Understanding these triggers helps you avoid surprises like queries returning stale data.

```
em.getTransaction().begin();

Item item = em.find(Item.class, 1L);
item.setName("Updated");           // dirty

// Trigger 1: flush fires before this query (same table)
List items = em.createQuery(
    "SELECT i FROM Item i", Item.class).getResultList();
// query now sees "Updated"

// Trigger 2: commit flushes remaining dirty entities
em.getTransaction().commit();
```

Key Rules

- Flush before JPQL query: only if the dirty entity's table overlaps the query's FROM clause.
- Flush before commit: always happens (unless `FlushMode.MANUAL`).
- Native SQL queries do NOT trigger auto-flush -- use `em.flush()` before them.
- Calling `em.refresh()` discards local changes and reloads from DB.
- `em.flush()` does NOT commit -- it only sends SQL; the transaction can still be rolled back.

Q1. Does Hibernate flush before every SELECT?

No. With `FlushMode.AUTO` it only flushes before a JPQL or Criteria query if Hibernate detects that dirty entities share the same table as the query's target.

Q2. What happens if you run a native SQL query before flushing?

Hibernate cannot analyse native SQL, so it does NOT auto-flush. You may read stale data. Always call `em.flush()` manually before native queries that need current state.

Q3. Can a flush be rolled back?

Yes. A flush sends SQL to the DB but the transaction has not committed. If the transaction is rolled back, all those SQL changes are undone.

Q4. What does `em.refresh()` do?

It discards the entity's current state and snapshot, reloading fresh data from the database. Any un-flushed changes to that entity are lost.

Q5. Is there a way to prevent flush before a specific query?

Yes. Temporarily set `FlushMode.COMMIT` (or `MANUAL`) on the session before the query, then restore it afterwards.

5 @DynamicUpdate

Only update changed columns, not all columns

By default, Hibernate generates a single static UPDATE statement that sets ALL columns every time an entity is flushed, even if only one field changed. This is efficient because the SQL can be pre-compiled and cached. However, in tables with many columns (or wide BLOB/CLOB columns), updating everything wastes bandwidth. Annotating the entity with **@DynamicUpdate** tells Hibernate to generate the UPDATE dynamically at runtime, including only the columns that actually changed.

```
import org.hibernate.annotations.DynamicUpdate;

@Entity
@DynamicUpdate          // only changed columns in UPDATE
public class Employee {
    @Id
    private Long id;
    private String name;
    private String department;
    private double salary;
    // getters + setters ...
}

// If only salary changes:
// Without @DynamicUpdate: UPDATE employee SET name=?,department=?,salary=? WHERE id=?
// With    @DynamicUpdate: UPDATE employee SET salary=? WHERE id=?
```

Key Rules

- @DynamicUpdate is a Hibernate-specific annotation, not part of the JPA standard.
- Without it, Hibernate uses a pre-compiled static UPDATE covering all columns.
- Use it when the entity has many columns or expensive column types (BLOB, CLOB).
- It has a small overhead: Hibernate must build the SQL at runtime instead of reusing a cached statement.
- Does NOT affect dirty-checking logic -- Hibernate still detects what changed.

Q1. What does @DynamicUpdate do?

It makes Hibernate generate an UPDATE statement at runtime containing only the columns that actually changed, instead of a static UPDATE covering every column.

Q2. Is @DynamicUpdate a JPA annotation?

No. It is Hibernate-specific (org.hibernate.annotations.DynamicUpdate). Using it makes your code less portable to other JPA providers.

Q3. When should you use @DynamicUpdate?

When entities have many columns, or large BLOB/CLOB columns, and you want to minimise the data sent to the database on partial updates.

Q4. Does @DynamicUpdate improve performance always?

Not always. For small entities it may be slower because the SQL cannot be cached. Benchmark before applying it broadly.

Q5. Does @DynamicUpdate change how dirty checking works?

No. Dirty checking still detects all changed fields. @DynamicUpdate only affects which columns appear in the generated UPDATE statement.

6 Explicit flush()

flush() vs relying on commit

Relying on the transaction commit to flush is the most common approach and works well for simple request-scoped transactions. However, there are scenarios where calling **em.flush()** explicitly is useful or necessary: when you need to check for constraint violations before commit, when using FlushMode.MANUAL, when running native SQL after JPQL changes, or in long-running batch jobs where you want to release DB locks early by flushing and clearing the L1 cache periodically.

```
em.getTransaction().begin();

for (int i = 0; i < 10000; i++) {
    Product p = new Product("Item" + i, i * 1.5);
    em.persist(p);

    if (i % 50 == 0) {
        em.flush(); // send SQL to DB now
        em.clear(); // free L1 cache memory
    }
}

em.getTransaction().commit(); // final flush + commit
em.close();
```

Key Rules

- em.flush() sends pending SQL to the database but does NOT commit the transaction.
- The transaction can still be rolled back after em.flush().
- Flushing + clearing periodically in batch jobs prevents OutOfMemoryError.
- em.flush() throws PersistenceException if a constraint violation is detected.
- Do not call em.flush() excessively in normal CRUD -- it adds round trips unnecessarily.

Q1. What is the difference between em.flush() and em.getTransaction().commit()?

flush() sends pending SQL to the database but keeps the transaction open. commit() calls flush() internally and then commits the transaction, making changes permanent.

Q2. Why flush then clear in a batch loop?

Flushing sends SQL to the DB; clearing evicts entities from the L1 cache, preventing OutOfMemoryError when inserting thousands of rows.

Q3. Can a constraint violation be caught after flush() before commit?

Yes. flush() triggers SQL execution, so DB-level constraint violations surface as a PersistenceException at flush time, not at commit.

Q4. Is calling em.flush() required before em.close()?

No. Closing the EntityManager without committing discards all changes. You need to commit or explicitly flush within an active transaction.

Q5. When is explicit flush() essential?

Before executing native SQL queries that depend on un-committed JPA changes, and in batch loops to control memory and database round trips.

7 Detached Entities and merge()

Re-attach detached entities to the persistence context

An entity becomes **detached** when the EntityManager that loaded it is closed, when `em.detach()` is called explicitly, or when `em.clear()` evicts everything. Detached entities are no longer tracked -- changes made to them are NOT dirty-checked and will NOT be flushed automatically. To persist changes made to a detached entity, you must call **`em.merge()`**, which copies the detached entity's state into a managed entity instance and returns the managed copy.

```
// Load and close -> entity becomes detached
EntityManager em1 = emf.createEntityManager();
Customer c = em1.find(Customer.class, 5L);
em1.close(); // c is now detached

c.setPhone("555-9999"); // change on detached entity (not tracked)

// New session: merge re-attaches
EntityManager em2 = emf.createEntityManager();
em2.getTransaction().begin();
Customer managed = em2.merge(c); // managed copy returned
// managed is tracked; c is still detached
em2.getTransaction().commit();
em2.close();
```

Key Rules

- Detached entities are NOT dirty-checked -- Hibernate ignores their changes.
- `em.merge()` copies state from the detached instance into a managed instance and returns it.
- After `merge()`, use the returned managed instance -- the original detached object is still detached.
- `merge()` may issue a SELECT first to load the existing DB state before merging.
- `em.persist()` on a detached entity throws `EntityExistsException` -- use `merge()` instead.

Q1. What is a detached entity?

An entity instance that was once managed but whose EntityManager has been closed or that was explicitly removed from the persistence context via `detach()` or `clear()`. Hibernate no longer tracks its changes.

Q2. What does `em.merge()` do?

It copies the state of the detached (or transient) entity into a managed entity associated with the current persistence context and returns that managed entity.

Q3. Should you use the original object or the result of `merge()`?

Always use the object returned by `merge()`. The original detached object remains detached and its subsequent changes will not be tracked.

Q4. Does `merge()` always hit the database?

Not always. If the entity is already in the L1 cache, Hibernate may use that. If not, it issues a SELECT to fetch the current state before merging.

Q5. What is the difference between `merge()` and `persist()`?

`persist()` makes a new transient entity managed. Calling `persist()` on a detached entity throws an exception. `merge()` handles both detached and transient entities by copying state, and is the correct choice for re-attaching modified detached objects.